



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 3

No Linealidad: Árboles de Decisión y Random Forest



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos

✓ Ya tenemos:

- ✓ Regresión lineal: predecir un **número**
- ✓ Regresión logística: predecir una **etiqueta**
- ✓ Métricas + **costo de negocio** y umbral
- ✓ Train/test, estandarización, binarización

Pero ambos modelos comparten un límite: por dentro son una recta.

▶▶ Hoy:

- Qué es la **no linealidad** y por qué el mundo real casi siempre es así
- Ver a la recta **fracasar** (las lunas)
- **Árboles de decisión**, explicados desde cero
- ★ **Random Forest** — y batir el récord de costo de la Clase 2

*Al final: los árboles también predicen números (un **medidor virtual de flujo**).*

¿Qué es la No Linealidad?

Cuando el paso deja de ser parejo

Recuerden: Lineal = el Paso Parejo

En la Clase 1 lo definimos: lineal es cuando el mismo paso en X produce siempre el mismo paso en Y.

✓ LINEAL (el paso vale igual)

Tanquear el carro: el galón 1 cuesta lo mismo que el galón 10.

Presión hidrostática: cada 100 m suman $\approx$ lo mismo.

✗ NO LINEAL (el paso cambia)

Abrir el choke de 20 % a 30 % **no** da lo mismo que de 70 % a 80 %.

El declive: el primer año el pozo cae muchísimo; el quinto, casi nada.

- **No lineal** = el efecto de una variable **depende de dónde estás**.
- Y a veces depende de **otra** variable: eso se llama **interacción** (el efecto del choke depende de la presión disponible).

El Mundo Real Rara Vez es una Recta

Piensen en su operación: ¿cuántas de estas relaciones son “parejas”?

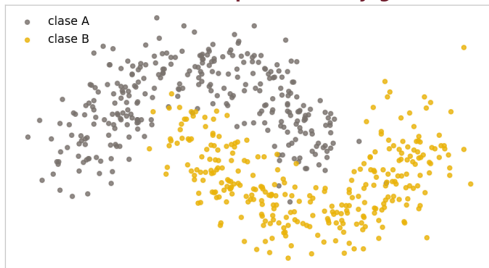
Relación	Por qué NO es lineal
Choke vs caudal	al inicio cada punto de apertura suma mucho; cerca del máximo, casi nada
Tiempo vs producción	la curva de declive: cae rápido y luego lento
Frecuencia de bomba vs caudal	sube hasta una meseta : el pozo no da más
Presión vs caudal (IPR)	la curva IPR es... una curva
Temperatura vs viscosidad	exponencial: unos grados cambian todo
GR vs litología	no es gradual: hay un umbral donde la roca cambia

💡 La recta es el modelo **más simple y explicable** — por eso empezamos ahí. Pero cuando la física es curva, **necesitamos modelos que doblen.**

Un Experimento: las Dos Lunas

Un problema de juguete, diseñado para ver el límite de la recta.

Las dos lunas: un problema de juguete



- Dos grupos con forma de **medialunas entrelazadas**.
- 👁 Cualquier persona los separa **a ojo**, sin pensar.
- ? Pregunta: ¿puede separarlos una **línea recta**?

Este dataset (`make_moons`) es el examen clásico de no linealidad en ML.

La Regresión Logística lo Intenta... y Fracasa

La logística solo sabe trazar una frontera recta. Es lo mejor que puede hacer:

La logística solo sabe trazar rectas (acc=0.86)



- ✗ Su recta corta las dos lunas por la mitad: se queda en **0.86 para siempre**.
- No es falta de datos ni de ajuste: es un límite **estructural**. La recta no puede doblar.
- ? ¿Qué modelo puede seguir la forma de las lunas? Eso construiremos hoy, **paso a paso**.

El Reto de Hoy

Batir el récord de la Clase 2

Volvemos a la Litología — a Propósito

Mismo problema de la Clase 2: ¿esta roca es reservorio (arenisca) o sello (lutita)?

- 📄 Mismo dataset FORCE 2020: **62 792** profundidades, 11 pozos, 5 registros.
- ⚖️ Mismo desbalance: **83 % lutita**, 17 % arenisca.
- ✅ ¿Por qué repetirlo? Para una comparación **justa**: mismos datos, mismo test, mismas métricas. **Solo cambia el modelo.**

El récord a batir (Clase 2):

Regresión logística, umbral 0.5:

- accuracy **0.944**
- recall **0.79** (se escapan 2 de cada 10 zonas)
- **563 zonas perdidas**, 323 cañoneos en seco
- costo total: **5 953**

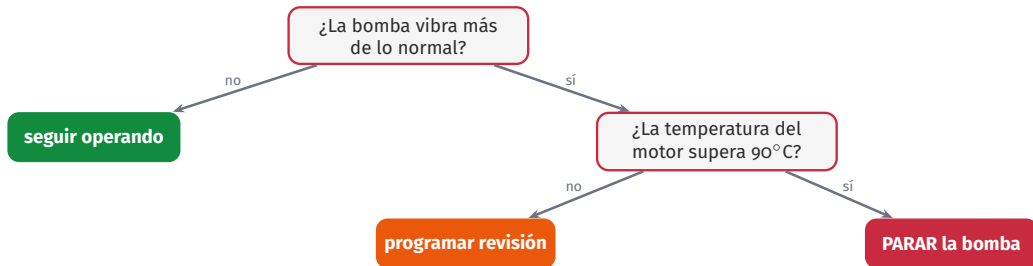
*La física de la litología tiene **umbrales e interacciones**: terreno ideal para modelos no lineales.*

Árboles de Decisión

Desde cero, paso a paso

¿Qué es un Árbol de Decisión? (Sin Datos Todavía)

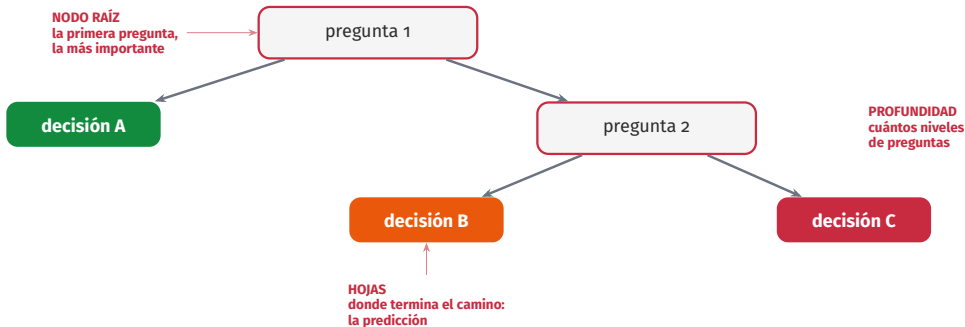
Es la forma en que ya deciden en el campo: una cadena de preguntas de sí/no.



- Cada pregunta **divide** los casos en dos; al final de cada camino hay una **decisión**.
- ✓ Así opera un supervisor con 20 años de experiencia. El árbol de ML hace **exactamente esto**.

La Anatomía: Raíz, Ramas y Hojas

El mismo diagrama, con los nombres técnicos que usaremos el resto de la clase.



💡 Este árbol tiene **profundidad 2**: máximo dos preguntas antes de decidir. Recuerden esa palabra — será **la perilla clave** del modelo.

Ustedes Ya Piensan en Árboles (con Datos)

En petrofísica, esas preguntas de sí/no tienen nombre: cutoffs.

- 🔗 “Si el GR $< 60 \rightarrow$ arena; si no $\rightarrow$ arcilla.”
- 🔗 Y los encadenan: “si es arena **y** la porosidad $> 15\%$ **y** la resistividad es alta $\rightarrow$ zona de interés.”
- ❓ Pero... ¿por qué 60 y no 55? ¿Por qué GR primero y no la porosidad?

La diferencia con el árbol de ML:

los cutoffs del intérprete salen de la **experiencia**. Los del árbol salen de **probar todos los cortes posibles en los datos** y quedarse con los que mejor separan.
Ni mejor ni peor: **verificable y repetible**.

Veamos exactamente **cómo elige** la máquina un corte.

¿Cómo Elige la Máquina la Pregunta?

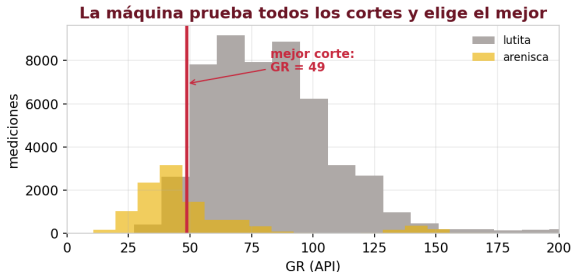
Fuerza bruta inteligente: prueba todos los cortes de todas las variables.

1. Toma una variable (p. ej. GR) y un corte candidato (p. ej. 55): divide los datos en dos grupos.
2. Mide qué tan **puros** quedaron los grupos: un grupo es puro si casi todos sus puntos son de la **misma clase**.
3. Repite con **cada corte posible de cada variable** y se queda con el que deja los grupos **más puros**.
4. Dentro de cada grupo, **vuelve a empezar**: busca la mejor pregunta para ese subgrupo. Así crece el árbol.

💡 “Puro” es la versión formal de lo que un intérprete busca a ojo: un corte donde **a un lado quede la arena y al otro la arcilla**.

El Primer Corte, con Nuestros Datos Reales

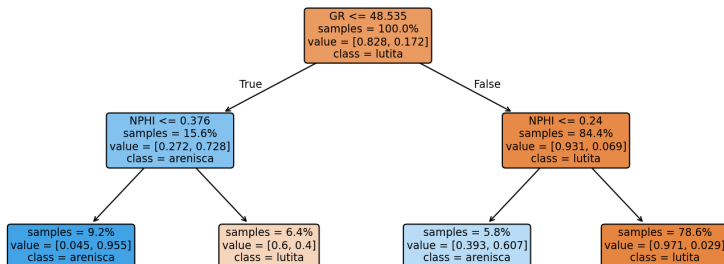
Dejamos que la máquina elija una sola pregunta sobre el GR. Esto encontró:



- ✓ El corte óptimo: **GR = 49**. Muy cerca del cutoff que un intérprete usaría (~50–60).
- ✓ Con **una sola pregunta**: accuracy **0.90** — ya le gana al “modelo tonto” (0.83).
- ➔ Pero el traslape no se resuelve con un corte: hay que **seguir preguntando**.

Encadenando Preguntas: el Árbol Crece

Con profundidad 2, la máquina eligió esto. Leámoslo juntos, nodo por nodo.

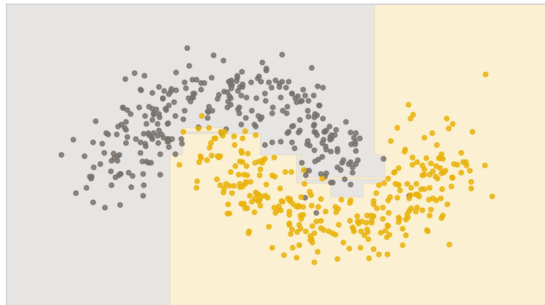


- **Raíz:** ¿ $GR \leq 49$? — la misma pregunta de la slide anterior.
- Luego refina con NPHI (la porosidad de neutrón) **en ambos lados**, con cortes **distintos** en cada lado.
- ✓ GR bajo + NPHI bajo → **arenisca**. GR alto → **lutita**. La física aprendida de los datos.

Ahora Sí: las Lunas con un Árbol

Cada pregunta corta el plano en dos. Muchas preguntas = una frontera de escalones.

El árbol dibuja escalones que siguen la forma (acc=1.00)



- ✓ Los escalones **siguen la forma** de las lunas: accuracy **1.00** donde la recta se quedó en 0.86.
- El árbol nunca “dobla” — pero acumula tantos cortes rectos que **aproxima cualquier frontera**.
- Igual que los escalones aproximaban la curva del declive.

En Código: el Mismo Patrón de Siempre

Cambia el modelo, no el flujo. Y un regalo: los árboles no necesitan estandarización.

Python

```
from sklearn.tree import  
    DecisionTreeClassifier  
  
arbol = DecisionTreeClassifier(  
    max_depth=5, random_state=42)  
arbol.fit(X_tr, y_tr)  
  
print(arbol.score(X_te, y_te))
```

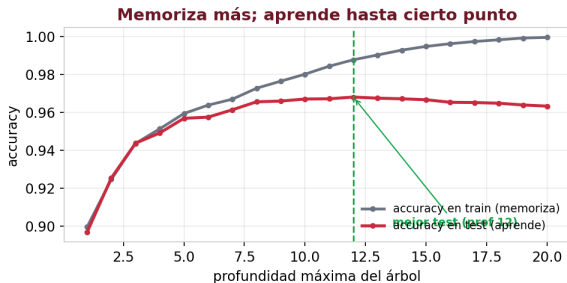
>_ Output

0.957

- ✓ Con 5 niveles: accuracy **0.957**, recall **0.84** — ya supera a la logística (0.944 / 0.79).
- max_depth: cuántos niveles de preguntas. **La perilla clave.**
- Los cortes comparan cada variable **consigo misma** → las escalas no importan.

La Tentación: ¿y si lo Dejamos Crecer sin Límite?

Más profundidad = más poder... hasta que empieza a memorizar.



- Sin límite, el árbol crea una hoja para **cada rincón** de los datos: accuracy **train** = **1.000**.
- ⚠ Pero en **test** deja de mejorar: aprendió el **ruido**, no el patrón.
- ✓ Es el **overfitting** de la Clase 1, ahora **visible en una curva**.

El Árbol Solo: Poderoso pero Indisciplinado

Dos debilidades que impiden confiar en un único árbol.

⚠ Memoriza

Si lo dejamos crecer, aprende el ruido de los datos de entrenamiento — lo acabamos de ver en la curva.

⚠ Es inestable

Cambien un puñado de datos y el árbol puede elegir **otra** primera pregunta — y cambiar entero. Un experto que cambia de opinión con cada informe.

La solución **no** es buscar un árbol mejor.

Es dejar de confiar en **uno**: usar **muchos árboles a la vez**.

Random Forest

La sabiduría de la multitud

La Idea: Preguntarle a 200 Ingenieros Distintos

Un experto se equivoca. El promedio de muchos expertos independientes se equivoca mucho menos.

- 🌲 **Random Forest** = bosque aleatorio: entrena **cientos de árboles**, cada uno un poco distinto.
- 👥 Para clasificar: cada árbol **vota** (¿arenisca o lutita?) y gana la mayoría.
- % El porcentaje de votos funciona como **probabilidad** → todo lo de la Clase 2 (umbral, costos) **sigue valiendo**.

Por qué funciona:

cada árbol memoriza **ruido distinto**. Al votar, los errores individuales **se cancelan** y queda lo que todos vieron: **la señal**.

Como pedirle la interpretación a 200 geólogos que estudiaron el campo con informes diferentes — y quedarse con el consenso.

El Truco: Obligarlos a Pensar Distinto

200 copias del mismo árbol no sirven de nada. La clave es la diversidad — y se fabrica con azar.

📊 Azar en los datos

Cada árbol entrena con una **muestra al azar** de las profundidades (con reemplazo).

Como si cada geólogo estudiara pozos distintos del mismo campo.

🔗 Azar en las variables

En cada corte, el árbol solo puede mirar un **subconjunto al azar** de los registros.

Así ninguno se obsesiona con el GR: algunos aprenden a usar NPHI, DTC...

- ✓ Resultado: árboles que se equivocan **en cosas diferentes** — la condición para que el voto funcione.
- ➔ Cada árbol individual crece **sin límite** (memoriza), pero el **promedio** de sus memorias es estable.

Random Forest en Código

Una línea distinta. El resto del flujo, idéntico.

Python

```
from sklearn.ensemble import  
    RandomForestClassifier  
  
bosque = RandomForestClassifier(  
    n_estimators=200, random_state=42)  
bosque.fit(X_tr, y_tr)  
  
print(bosque.score(X_te, y_te))
```

>_ Output

0.979

- `n_estimators`: cuántos árboles. 200 es un buen punto de partida.
- ✓ Accuracy **0.979**, recall **0.93**: de perder 2 de cada 10 zonas... a perder **menos de 1**.
- Sin estandarizar, sin transformar: los árboles trabajan con los datos tal cual.

No Basta la Accuracy: Abrir la Caja

Como en la Clase 2: la accuracy resume... y esconde. Miremos los cuatro resultados.

(test, umbral 0.5)	Logística (C2)	Random Forest
Zonas perdidas (FN) — <i>el error caro</i>	563	203
Cañoneos en seco (FP)	323	128
Reservorios encontrados (TP)	2 132	2 492
Sellos correctos (TN)	12 680	12 875

- ✓ El bosque **mejora las cuatro celdas a la vez**: no es un trade-off, es un modelo genuinamente mejor.
- Las **360 zonas** que la logística perdía y el bosque encuentra son reservas que antes quedaban en el subsuelo.

Las Métricas de la Clase 2, Aplicadas al Bosque

Mismas preguntas de siempre: ¿cuánto encuentro? ¿cuánto me creen cuando digo que sí?

Métrica	Logística	R. Forest	Lectura
Accuracy	0.944	0.979	ambas “se ven bien”
Recall	0.791	0.925	de perder 2 de 10 zonas... a menos de 1
Precision	0.868	0.951	cuando dice reservorio, casi siempre acierta
F1	0.828	0.938	el balance de ambas

⚠ Fíjense en la accuracy: sube “solo” 3.5 puntos. El **recall** sube **13**. Otra vez: la accuracy es la métrica que **menos** cuenta la historia.

✓ Regla de la Clase 2 intacta: **nunca** reportar una sola métrica.

El Marcador Completo

Todos los modelos del módulo, mismo test, mismas métricas y el costo de la Clase 2.

Modelo	Acc	Recall	Prec	FN	Costo
Logística (Clase 2)	0.944	0.791	0.868	563	5 953
Árbol prof 5	0.957	0.837	0.905	438	4 617
Árbol prof 10	0.967	0.873	0.932	342	3 593
Random Forest	0.979	0.925	0.951	203	2 158

→ Noten la **escalera**: cada nivel de flexibilidad (recta → árbol → bosque) baja el costo.

? ¿Y si además usamos la **perilla del umbral** de la Clase 2?

La Perilla Sigue Ahí: Umbral en el Bosque

El bosque también da probabilidades (% de votos) → todo lo de la Clase 2 aplica.

Python

```
proba = bosque.predict_proba(X_te)[: , 1]

# costo de la Clase 2: FN=10, FP=1
pred = (proba >= 0.11).astype(int)
```

>_ con umbral 0.11

```
zonas perdidas (FN):      42
cañoneos en seco (FP): 730
recall:                0.984
costo total:           1,150
```

🏆 De 5 953 (logística, 0.5) a 1 150: **81 % menos costo.**

→ Aceptamos 407 cañoneos en seco **más** que la logística... a cambio de rescatar **521 zonas productivas.**

✓ Recall **0.98**: de cada 100 zonas, encontramos 98.

Cómo Reportarlo al que Decide

El gerente no habla en F1. La regla de la Clase 2, aplicada al resultado de hoy.

✗ Así NO:

"Migramos de regresión logística a un ensamble de 200 árboles con umbral optimizado; el F1 subió de 0.83 a 0.94 y el recall a 0.98."

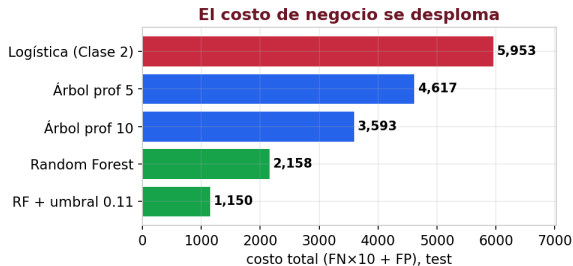
✓ Así SÍ:

*"Con el nuevo modelo dejamos de perder **521 zonas productivas** al año de interpretación, a cambio de ~400 pruebas adicionales que salen secas. En el balance de costos que nos dio yacimientos, eso **reduce el costo total 81 %**."*

- Las métricas son **para ustedes**; las **consecuencias** son para quien decide.
- ✓ Eso — no el modelo — es lo que hace que el proyecto se apruebe.

El Marcador: Mismo Problema, Mismo Test

Todos los modelos, evaluados con el costo de la Clase 2 (perder zona = $10 \times$ cañoneo seco).



✓ El bosque baja el costo de **5 953** a **2 158**: **64 % menos** que la logística.

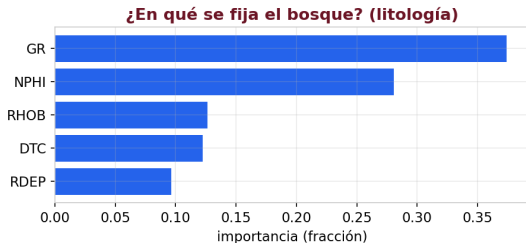
🏆 Y aplicando el **umbral por costo** de la Clase 2: **1 150**. Un **81 %** menos.

★ Zonas perdidas: de **563** (logística) a **42**.

💡 Las dos clases se suman: **mejor modelo** (hoy) $\times$ **mejor decisión** (Clase 2). Ninguna reemplaza a la otra.

¿En Qué se Fija el Bosque?

El bosque no da coeficientes, pero sí dice cuánto usó cada variable.



- 🏆 GR domina (37 %) — coherente con que el mejor corte univariado era GR = 49.
- ✓ NPHI segunda (28 %): la misma que el árbol pequeño eligió para refinar.
- ⚠️ Ojo: importancia $\neq$ causalidad. Dice qué **usó** el modelo, no qué **hace** que la roca sea arena.

El Precio: Poder vs Explicabilidad

Nada es gratis. Lo que ganamos en precisión lo pagamos en transparencia.

	Logística	Árbol	R. Forest
Captura no linealidad	no	sí	sí
Resiste overfitting	sí	no	sí
Se explica completo	sí (fórmula)	sí (diagrama)	no (200 árboles)
Necesita estandarizar	sí	no	no

- Del bosque solo podemos explicar las **importancias**, no el camino de cada predicción individual.
- ✓ La práctica profesional: **empezar simple** (línea base explicable) y **escalar** solo si el negocio necesita la precisión extra.

Su Turno: Batir a la Logística

🕒 15 min

En el notebook, con el dataset de litología ya cargado:

1. Entrenen un `DecisionTreeClassifier` con `max_depth=3`. Grafíquenlo con `plot_tree`: ¿qué preguntas eligió?
2. Prueben `max_depth = 2, 5, 10` y sin límite. Comparen accuracy en **train** y en **test**: ¿dónde empieza a memorizar?
3. Entrenen un `RandomForestClassifier` y saquen la **matriz de confusión**: ¿cuántas zonas perdidas quedaron?
4. Con `predict_proba` y el costo de la Clase 2 ($FN = 10 \times FP$), encuentren el **umbral óptimo** del bosque. ¿Baja de 2 000?

Bonus: También Predicen Números

Un medidor virtual de flujo

¿Cuánto Está Produciendo Cada Pozo, Ahora?

Los árboles y bosques también hacen regresión. Un caso real para demostrarlo.

- 🔧 Medir el caudal de **un** pozo exige desviarlo al **separador de prueba**: se hace cada tantos días, no a diario.
- ⚡ Lo que **sí** está siempre: los sensores de presión, temperatura y choke.
- 💡 **La idea**: estimar la producción desde esos sensores — un *medidor virtual de flujo*.

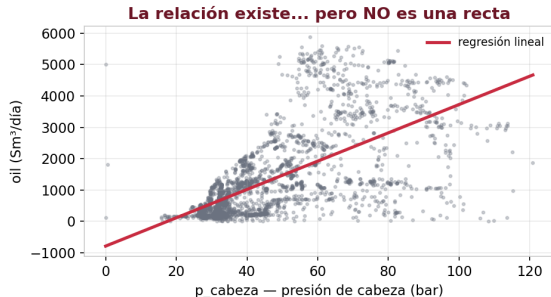
El dataset (en el repo):

operación diaria de **5 pozos** de Volve: 7 862 días con `p_fondo`, `p_cabeza`, `t_cabeza`, `choke`, `dp_choke`, `horas` → `oil` medido.

Los virtual flow meters son un producto real de la industria.

Otra Vez la Física Curva

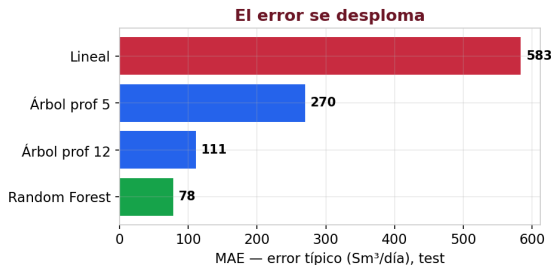
La relación más fuerte (p_cabeza vs oil) tiene forma de codo. La recta no puede.



- ✗ Regresión lineal con las 6 variables:
 $R^2 = 0.60$, error típico **583** Sm³/día
— casi la mitad de la producción media (1264).
- ➔ No es que el problema sea impredecible: es que **la recta no dobla**.

El Bosque Regresor: el Error se Desploma

RandomForestRegressor: mismos árboles, pero cada hoja predice un número (el promedio de sus días).



- ✓ De **583** a **78** Sm³/día de error: **7.5 veces menos**. $R^2 = 0.98$.
- ✓ De inutilizable a **operativo**: el medidor virtual funciona.
- Y a diferencia de la recta del declive (Clase 1), el bosque **nunca predice producción negativa**: no inventa fuera de lo que vio.

Su Turno: el Medidor Virtual

🕒 10 min

En el notebook, con el dataset de operación (op):

1. Explore: scatter de p_cabeza vs oil. ¿Ven el codo?
2. Entrenen la **regresión lineal** (línea base) y midan R^2 y MAE en test.
3. Entrenen un RandomForestRegressor y comparen las mismas métricas.
4. Saquen las **importancias**: ¿qué sensor domina? ¿Tiene sentido físico?
5. Grafiquen **predicho vs real** del bosque en el test.

Cierre

Lo que se llevan hoy

Lo Que Aprendimos Hoy

Conceptos

- ✓ No linealidad: el paso deja de ser parejo
- ✓ La recta fracasa (las lunas)
- ✓ Árbol = preguntas encadenadas (raíz, hojas, profundidad)
- ✓ Cómo la máquina elige cada corte (pureza)
- ✓ Overfitting visible: train 1.000 vs test
- ✓ Random Forest: diversidad + voto
- ✓ Importancias ($\neq$ causalidad); poder vs explicabilidad

Herramientas

- ✗ `DecisionTreeClassifier / Regressor`
- ✗ `RandomForestClassifier / Regressor`
- ✗ `plot_tree, feature_importances_`
- ✗ `max_depth, n_estimators`

El número del día:

costo de la litología: logística **5 953** → bosque + umbral **1 150**.

Mejor modelo (hoy) × **mejor decisión** (Clase 2).

Lo Que Sigue

Ya tienen modelos lineales y no lineales. Falta afinar cómo los validamos y ajustamos.

- ➔ **Validación seria** — validación cruzada, y el detalle que dejamos anotado: evaluar con **pozos completos** por fuera.
- ➔ **Afinar las perillas** — búsqueda sistemática de hiperparámetros (`max_depth`, `n_estimators...`).
- ➔ **Multiclase** — las 11 litologías reales del FORCE 2020, con su matriz de penalización de la competencia.

La pregunta de siempre sigue mandando: ¿qué decisión habilita este modelo y cuánto cuesta equivocarse?

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

SLB Ecuador · UDLA · 2026